Overview

Now that the Wild Rydes application has been converted from .NET Framework 4.8.1 to .NET 8, it's time to deploy it. The converted application includes a Linux-based multi-stage Dockerfile, making it ready for containerized deployment.

In this section, you'll set up a SQL Server database, build the Docker image, run the application locally, and verify it works.

EXERCISE 1: SET UP SQL SERVER

The modernized application requires SQL Server. We'll use Docker to run a SQL Server container and use Kiro CLI to set it up.

1. Ensure you are in the wildrydes directory

```
cd /workshop/wildrydes
```

2. Start Kiro CLI

```
kiro-cli chat
```

3. Ask Kiro to pull and run SQL Server in Docker

```
Pull and run Microsoft SQL Server 2022 in Docker with the following:
- Use the mcr.microsoft.com/mssql/server:2022-latest image
- Set SA password to "YourStrong!Passw0rd"
- Accept the EULA
- Map port 1433
- Name the container "mssql"
- Run in detached mode
- All docker commands must use sudo
- Wait for the container to be healthy, then create a database called "wildrydes-net"
```

4. Once Kiro finishes setting up SQL Server, exit the session

```
/quit
```

5. Verify SQL Server is running

```
sudo docker ps
```

You should see the `mssql` container running on port 1433.

EXERCISE 2: UPDATE THE CONNECTION STRING

1. Update `appsettings.json` to connect to the SQL Server container

```
kiro-cli chat
```

```
Update sourceCode/wildrydes.net/appsettings.json to set the DefaultConnection string to:
"Server=host.docker.internal,1433;Database=wildrydes-net;User Id=sa;Password=YourStrong!Passw0rd;TrustServerCertificate=true;"
```

2. Exit Kiro CLI

```
/quit
```

3. Verify the connection string was updated

```
cat sourceCode/wildrydes.net/appsettings.json
```

EXERCISE 3: FIX CONVERSION LAYOUT ISSUES WITH KIRO CLI

The ATX conversion preserved functionality but introduced some visual layout issues during the framework migration. Let's use Kiro CLI to fix them.

1. Start Kiro CLI

```
kiro-cli chat
```

2. Ask Kiro to fix the layout issues

```
Fix the following conversion layout issues in the wildrydes.net ASP.NET Core app under sourceCode/wildrydes.net/:

1. In Views/Home/Index.cshtml - fix the missing closing </div> tag for the <div class="row"> containing the three unicorn sections
2. In Views/Home/Index.cshtml - add class="img-fluid" to the three unicorn images (mr-unicorn-one.png, mr-unicorn-two.png, mr-unicorn-three.png)
3. In Views/Shared/_Layout.cshtml - change mr-logo-white.png to mr-logo-black.png so the logo is visible on the light background
4. In wwwroot/Content/Site.css - fix the background image path from url("images/background.png") to url("/Content/Images/background.png")
5. In wwwroot/Content/Site.css - remove the max-width: 200px constraint on input, select, textarea

Use sudo for any docker commands.
```

3. Once Kiro finishes, exit the session

```
/quit
```

EXERCISE 4: APPLY DATABASE MIGRATIONS

The converted application uses EF Core 8, but the database tables don't exist yet. We'll use Kiro CLI to create and apply the initial migration.

1. Start Kiro CLI

```
kiro-cli chat
```

2. Ask Kiro to set up EF Core migrations

```
For the .NET 8 project at /workshop/wildrydes/sourceCode/wildrydes.net/:
1. Install the dotnet-ef CLI tool globally
2. Add the Microsoft.EntityFrameworkCore.Design package version 8.0.11
3. Create an Initial EF Core migration called "InitialCreate"
4. Apply the migration to create the database tables

Note: The appsettings.json uses host.docker.internal for the SQL Server connection, but dotnet ef runs on the host where localhost resolves correctly. Temporarily update
```

3. Once Kiro finishes, exit the session

```
/quit
```

EXERCISE 5: BUILD AND RUN THE APPLICATION

1. Navigate to the source code directory

```
cd /workshop/wildrydes/sourceCode
```

2. Build the Docker image using the converted Dockerfile

```
sudo docker build --no-cache -t wildrydes-net8 -f wildrydes.net/Dockerfile .
```

What's happening
The multi-stage Dockerfile:

1. Uses `mcr.microsoft.com/dotnet/sdk:8.0` to restore, build, and publish the application
2. Uses `mcr.microsoft.com/dotnet/aspnet:8.0` (Linux-based) as the runtime image
3. Exposes port 8080

3. Verify the image was built

```
sudo docker images wildrydes-net8
```

Expected result:

```
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
wildrydes-net8:latest  xxxxxxxxxxxx      xxxMB          xxxMB
```

EXERCISE 6: RUN THE APPLICATION

1. Run the container

```
sudo docker run -d -p 5000:8080 --name wildrydes --add-host=host.docker.internal:host-gateway wildrydes-net8
```

Key flag
`--add-host=host.docker.internal:host-gateway` lets the container resolve `host.docker.internal` to the host machine where the SQL Server container's port 1433 is mapped.

2. Verify the container is running

```
sudo docker ps
```

3. Test the application

```
curl -s http://localhost:5000 | head -20
```

You should see HTML output from the Wild Rydes home page.

Note
The application is accessible locally via `curl`, but not yet from your browser since the EC2 security group does not allow inbound traffic on port 5000. We'll fix that in the next exercise.

EXERCISE 7: ACCESS THE APPLICATION FROM YOUR BROWSER

To access the application from your browser, we need to find the server's public IP and open port 5000 in the security group. Let's use Kiro CLI to do this.

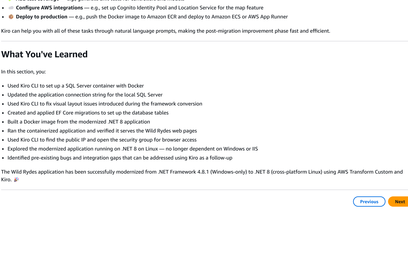
1. Start Kiro CLI

```
kiro-cli chat
```

2. Ask Kiro to find the public IP and open the security group

```
Find the public IP of this EC2 instance and update its security group to allow inbound TCP traffic on port 5000 from anywhere (0.0.0.0/0). Then tell me the URL I can use to
```

3. Kiro will output the URL. Open it in your browser to see the modernized Wild Rydes application.



4. Exit Kiro CLI

```
/quit
```

EXERCISE 8: EXPLORE THE MODERNIZED APPLICATION

Open the application URL in your browser and walk through the key features to verify the conversion works.

1. **Home page** — Navigate to the root URL

   You should see the Wild Rydes landing page with the unicorn images, navigation bar, and styled layout.

2. **Register a new user** — Click **Register** in the navigation bar

   - Enter an email and password
   - Click **Create**
   - You should be redirected to the home page after successful registration

3. **Login** — Click **Login** in the navigation bar

   - Enter the email and password you just registered
   - Click **Login**
   - You should be redirected to the home page with the session active

4. **View unicorns** — Click **Our Unicorns** in the navigation bar

   - You should see the unicorn listing page (empty initially since no seed data is loaded)

5. **View ride history** — Click **Ride History** in the navigation bar (visible after login)

   - This page shows your ride history (empty for now)

6. **Logout** — Click **Logout** in the navigation bar

   - You should be redirected to the home page with the session cleared

Known Limitations
The original source code contains some pre-existing bugs and incomplete integrations that were preserved during the platform migration (functional equivalence):

- **Unicorn creation** — The Create form is missing the `Color` field, which is `[Required]` on the model. Submissions silently fail validation.
- **AWS integrations** — The **Request a Ride** map feature requires AWS Cognito and Location Service credentials (`<CALACT_WITH_>` placeholders) which are not configured in this workshop.
- **Other issues** — See the technical debt report from the codebase analysis for the full list of pre-existing issues.

Next Steps: Fix and Improve with Kiro

The ATX transformation successfully migrated the platform from .NET Framework 4.8.1 to .NET 8. However, as with any real-world modernization, the converted application still has pre-existing bugs, security gaps, and areas for improvement.

We strongly recommend using Kiro CLI or Kiro IDE as the next step to:

- 🔧 **Fix pre-existing bugs** — e.g., add the missing `Color` field to the Unicorn Create form and `[Bind]` attribute
- 🔒 **Address security issues** — e.g., replace SHA256 password hashing with bcrypt/PBKDF2, add CSRF protection to `CreateSide`, harden session cookies
- ⚡ **Improve architecture** — e.g., extract business logic into service layers, add dependency injection for AWS services
- ✅ **Add test coverage** — e.g., generate unit tests for controllers and models
- ☁️ **Configure AWS integrations** — e.g., set up Cognito Identity Pool and Location Service for the map feature
- 🚀 **Deploy to production** — e.g., push the Docker image to Amazon ECR and deploy to Amazon ECS or AWS App Runner

Kiro can help you with all of these tasks through natural language prompts, making the post-migration improvement phase fast and efficient.

What You've Learned

In this section, you:

- Used Kiro CLI to set up a SQL Server container with Docker
- Updated the application connection string for the local SQL Server
- Used Kiro CLI to fix visual layout issues introduced during the framework conversion
- Created and applied EF Core migrations to set up the database tables
- Built a Docker image from the modernized .NET 8 application
- Ran the containerized application and verified it serves the Wild Rydes web pages
- Used Kiro CLI to find the public IP and open the security group for browser access
- Explored the modernized application running on .NET 8 on Linux — no longer dependent on Windows or IIS
- Identified pre-existing bugs and integration gaps that can be addressed using Kiro as a follow-up

The Wild Rydes application has been successfully modernized from .NET Framework 4.8.1 (Windows-only) to .NET 8 (cross-platform Linux) using AWS Transform Custom and Kiro. 🎉